

Feature Regularisation of a Tree Encoding under a Logistic-Regression Head

TKCE project
Experiment report

26 September 2026

Summary. With the classifier fixed to a scikit-learn logistic regression, twenty-one runs covering eighteen distinct feature-regularisation configurations were carried out on the *credit* dataset. **Test AUC is essentially invariant across all of them** (range 0.8424–0.8464 against a baseline of 0.8457), while the **train-minus-validation gap falls by a factor of more than ten** (0.0972 down to 0.0092). The most efficient configuration keeps only the three shallowest tree layers: **700 of 5,401 bits**, test AUC 0.8460, overfitting gap 0.0240, and a fit that is roughly fifteen times faster. **Depth-targeted noise gave no advantage over uniform noise of the same budget.**

1 Goal of the experiment

A previous experiment established that a plain logistic regression on the out-of-bag-honest tree encoding reaches test AUC 0.8467 on *credit*, against a LightGBM ceiling of 0.8453 and 0.8475 for a 2.44-million-parameter TabResNet. The neural network therefore contributed nothing measurable, and the result lives entirely in the features.

The classifier was consequently fixed to a logistic regression, which is the case of interest: overfitting is far milder than in the network, and the tree-encoding features already perform on par with the gradient-boosted baselines. All remaining experimentation moves to the *features*. This report asks four questions, following the directions set for this round:

1. Does adding noise to the encoding, at various levels, reduce overfitting, and at what cost in accuracy?
2. Does noise that *increases with the depth* of the split outperform uniform noise of the same total budget? Deeper splits are fitted on fewer training rows and should be the least trustworthy.
3. What happens when the deepest layers are deleted outright, and then the remaining layers are perturbed with a depth-increasing rate?
4. How much of the encoding is actually needed, tested by keeping only the first three layers (depths 0, 1 and 2)?

2 Dataset

Property	Value
Dataset	<i>credit</i> , OpenML task 361055 (Grinsztajn tabular benchmark)
Task	binary classification: serious payment delinquency within two years
Rows used	16000, split 11200 train / 2400 validation / 2400 test
Split	stratified random, fixed seed 0
Features	10 numerical (age, income, debt ratio, credit-line and late-payment counts, ...)
Target balance	approximately 50/50 by the benchmark's preprocessing
Preprocessing	features standardised using training-split statistics only
Known leakage	none; this dataset is not on the leaky-dataset list

Reference baselines fitted on the raw features of the same split:

Model	Test AUC	
LightGBM	0.8453	← the ceiling used throughout
XGBoost	0.8375	
Random Forest	0.8370	

3 Architecture

The model has two stages. The first is fixed and fitted once; the second is the only thing that learns.

Stage 1: the tree encoding (fixed). A random forest of 100 trees with `max_depth = 6` and `min_samples_leaf = 5` is fitted on the training split. Every *internal* node of every tree contributes one binary feature, called a *bit*: it is 1 if the row satisfies that node’s test $x_f \leq t$ and 0 otherwise. Concatenating over all nodes of all trees gives **5,401 bits**. The forest is never updated afterwards; it is a feature map, not a predictor.

Depth of split	0	1	2	3	4	5	total
Bits	100	200	400	780	1,455	2,466	5,401
Share of encoding	1.9%	3.7%	7.4%	14.4%	26.9%	45.7%	100%
Median training rows per split	$\approx 2,355$ for depths 0–2			≈ 133 –208 for depths 3–5			

Table 1: Almost half the encoding sits in the single deepest layer, and those splits are fitted on an order of magnitude fewer rows than the shallow ones. This distribution is why deletion is specified by *layer* rather than by a percentage.

Out-of-bag honesty. The forest was fitted using the training labels, so a training row’s bits from trees whose bootstrap sample *contained* that row are contaminated: those splits were partly carved to classify that very row. Test rows have no such trees. To remove this asymmetry, each training row keeps only the bits of trees for which it was out-of-bag (on average **36.8 of 100** trees), the rest are zeroed, and the survivors are rescaled by $T/|\text{OOB}|$ so expected magnitudes match. Validation and test rows use all 100 trees. This protocol is held fixed in every arm of this report.

Stage 2: the head (the only trained component).

$$\Pr(y = 1 \mid b) = \sigma(w^\top b + c), \quad \sigma(z) = \frac{1}{1 + e^{-z}},$$

a scikit-learn `LogisticRegression`: one weight per bit, *no hidden layer and no nonlinearity*. There is nothing between the encoding and the decision.

4 Training strategy

A convex fit, so no result can be blamed on the optimiser. The objective

$$\min_{w,c} \frac{1}{2} \|w\|_2^2 + C \sum_{i=1}^n \log \left(1 + \exp \left(-y_i (w^\top b_i + c) \right) \right)$$

is convex, and the solver (L-BFGS, `max_iter = 2000`) returns the global optimum. There is no learning rate, no epoch count, no batch size and no initialisation anywhere in this experiment.

The penalty is held constant at $C = 0.01$. C multiplies only the data-fitting term, so a *small* C means *strong* regularisation; in the textbook form $\sum \text{loss} + \lambda \|w\|^2$ this is $\lambda = 1/(2C)$. It is deliberately **not** re-tuned per arm: doing so would leave any score change attributable either to the features or to a differently tuned penalty. Holding it fixed makes every difference in the tables attributable to the features alone. For reference, an earlier sweep found $C = 0.001$ better by 0.0008 on validation, which is far inside the measurement noise.

How noise is applied to a convex solver. A mini-batch trainer can draw fresh flips at every batch, which is what makes noise a *regulariser* rather than one corrupted dataset. A convex solver sees a single fixed matrix, so the equivalent used here is to stack **three independently flipped copies** of the training rows and fit on all of them together (33600 rows), which approximates minimising the loss *averaged over the noise distribution*:

$$\min_{w,c} \mathbb{E}_{\text{flips}} [\mathcal{L}(w, c; \tilde{b})] \approx \frac{1}{K} \sum_{k=1}^K \mathcal{L}(w, c; \tilde{b}^{(k)}), \quad K = 3.$$

Order of operations, and what evaluation sees. Flips are applied to the *hard* 0/1 bits and the out-of-bag mask is applied afterwards, matching the earlier mini-batch implementation exactly. **Validation and test always use clean, unflipped, all-trees bits.** A deleted bit is removed as a column and its flip probability is forced to zero, so a flip can never resurrect a feature that was deleted on purpose.

How overfitting is measured. The reported gap is

$$\text{gap} = \text{AUC}_{\text{train}}^{\text{clean}} - \text{AUC}_{\text{validation}},$$

where the training term is evaluated on *clean* training bits, not on the flipped matrix the model was fitted to. Scoring a noisy arm against its own corrupted inputs makes the gap incomparable between arms and wrongly suggests that noise increases overfitting; this correction was made before the runs reported here.

5 Experimental setup

Four kinds of intervention, which compose. In all of them d denotes the depth of a split and $d_{\text{max}} = 5$ is the deepest internal layer.

Intervention	Definition	Question
Uniform noise	$p(d) = p$ for every bit	noise at various levels
Depth ramp	$p(d) = P \cdot d/d_{\text{max}}$; roots never flip	does more noise deeper help?
Deletion	drop all bits at the named depths	are the deep layers needed?
Combined	delete the deepest layers, then ramp the rest	the combined design

Matched budgets. For a ramp with parameter P , the average flip rate over all 5401 bits works out to $\bar{p} = \sum_d n_d p(d)/N = 0.796 P$ under this bit distribution. Every ramp is therefore paired with a *uniform* arm carrying the same average rate, so the *shape* of the noise can be separated from its *amount*. Without that pairing a ramp result cannot be interpreted.

Bits only. Every arm uses the tree bits alone, with no raw features. Including the ten raw columns would give the model an unperturbed shortcut (they reach 0.7785 by themselves under the same head), which would mask the effect of the noise. One **x+tree** control is reported for continuity.

Scope. Twenty-one runs were executed, covering eighteen distinct feature configurations; two configurations appear once but serve as both a noise level and a matched control, and three were refitted for stability.

Stability. Because the flips are random, three arms were refitted three times with fresh noise. The spread of test AUC was 0.0001–0.0002, so single runs are reliable with respect to the noise draw.

6 Results

Arm	Depths kept	Bits	$\bar{p}$	Train	Val	Test	Gap
<i>Baseline</i>							
all bits, no perturbation	0–5	5,401	0.0000	0.9465	0.8492	0.8457	0.0972
<i>Uniform noise at various levels</i>							
uniform $p = 0.05$	0–5	5,401	0.0500	0.9212	0.8490	0.8455	0.0722
uniform $p = 0.08$	0–5	5,401	0.0800	0.9100	0.8490	0.8456	0.0611
uniform $p = 0.10$	0–5	5,401	0.1000	0.9038	0.8487	0.8457	0.0551
uniform $p = 0.16$	0–5	5,401	0.1600	0.8908	0.8488	0.8460	0.0420
uniform $p = 0.20$	0–5	5,401	0.2000	0.8845	0.8487	0.8459	0.0359
uniform $p = 0.40$	0–5	5,401	0.4000	0.8587	0.8457	0.8433	0.0130
<i>Depth-increasing noise</i>							
ramp $P = 0.1$	0–5	5,401	0.0796	0.9109	0.8487	0.8457	0.0622
ramp $P = 0.2$	0–5	5,401	0.1592	0.8921	0.8479	0.8442	0.0442
ramp $P = 0.5$	0–5	5,401	0.3979	0.8548	0.8456	0.8424	0.0092
<i>Deleting the deepest layers</i>							
drop depth 5	0–4	2,935	0.0000	0.9162	0.8497	0.8464	0.0665
drop depths 4,5	0–3	1,480	0.0000	0.8898	0.8497	0.8460	0.0401
keep depths 0–2 only	0–2	700	0.0000	0.8737	0.8497	0.8460	0.0240
<i>Deletion combined with a depth ramp</i>							
drop 5 + ramp 0.2	0–4	2,935	0.0678	0.8854	0.8490	0.8458	0.0364
drop 4,5 + ramp 0.2	0–3	1,480	0.0247	0.8771	0.8491	0.8459	0.0280
drop 4,5 + ramp 0.2 rescaled	0–3	1,480	0.0412	0.8718	0.8482	0.8451	0.0236
keep 0–2 + ramp 0.2	0–2	700	0.0185	0.8635	0.8473	0.8437	0.0162
<i>Controls</i>							
x+tree (bits + 10 raw features)	0–5	5,411	0.0000	0.9469	0.8494	0.8454	0.0975
LightGBM on raw features	—	—	—	—	—	0.8453	—

Table 2: All eighteen distinct configurations (21 runs in total; three arms were refitted for stability, and two configurations served double duty as matched controls). $\bar{p}$ is the average flip probability over all 5401 bits. Gap is train AUC minus validation AUC with the training term measured on clean bits. Red marks the five arms that fall below the LightGBM ceiling; four of the five involve a depth ramp.

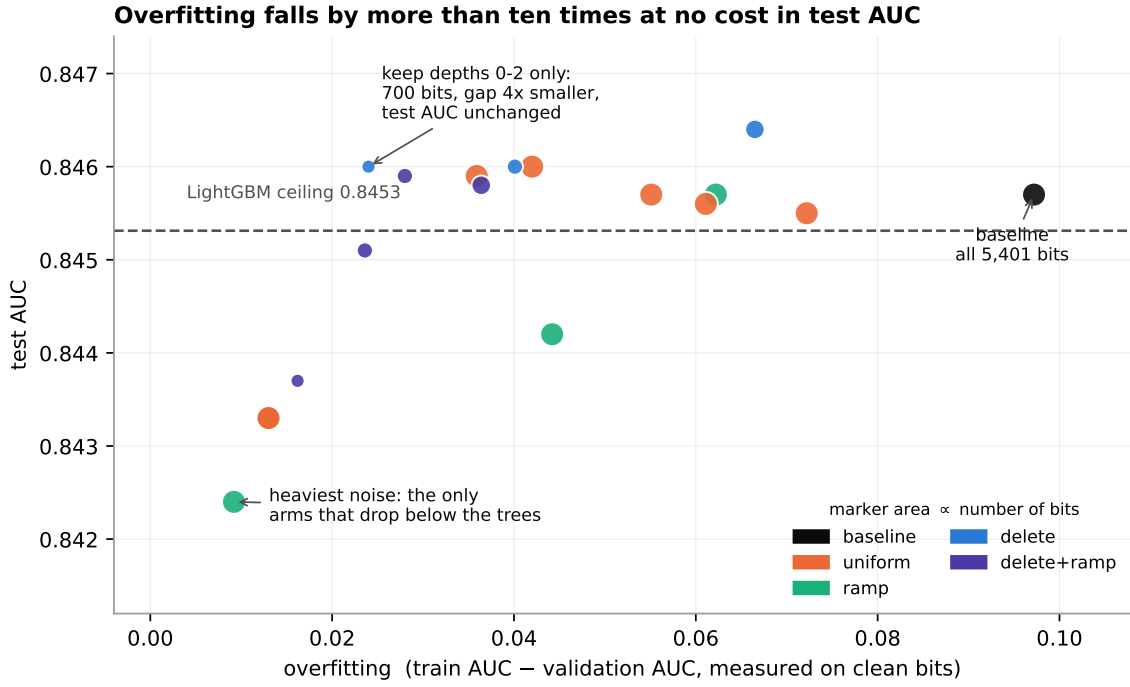


Figure 1: Every configuration sits in a narrow horizontal band while the overfitting gap varies by a factor of more than ten. Marker area is proportional to the number of bits used. The arms that fall below the tree ceiling are concentrated at the low-gap end, where the penalty has begun to suppress signal rather than memorisation.

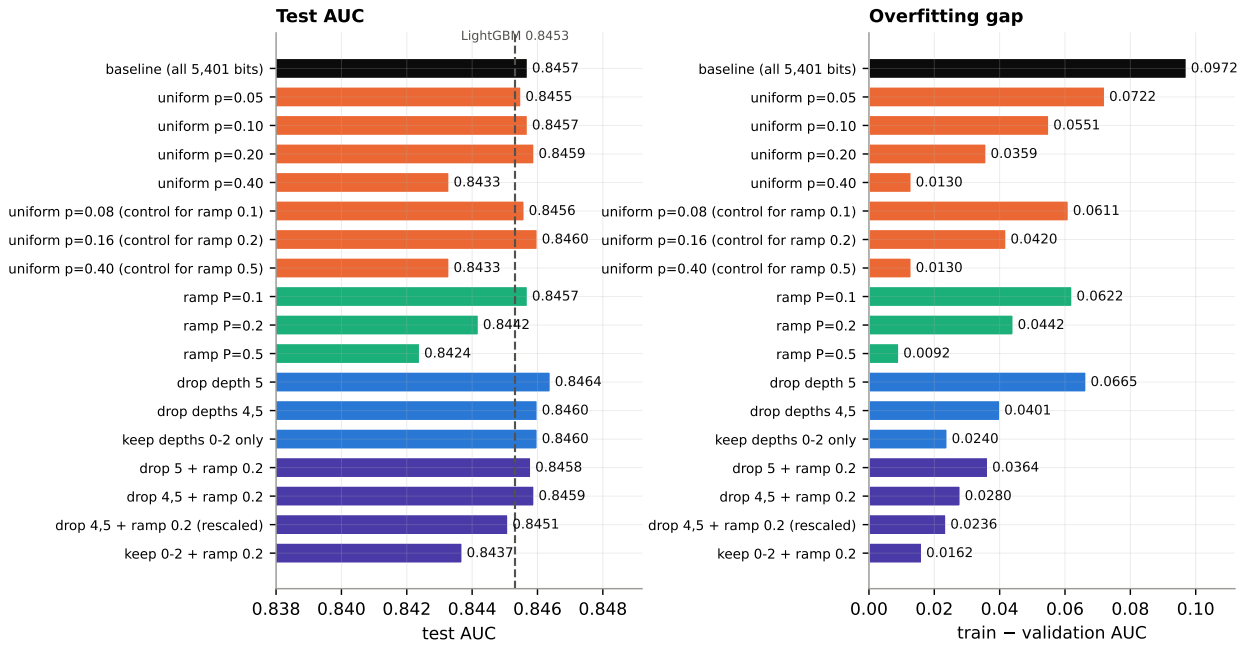


Figure 2: The same results as bars. Left: test AUC, with the LightGBM ceiling dashed. Right: the overfitting gap. The left panel is nearly flat; the right panel spans an order of magnitude.

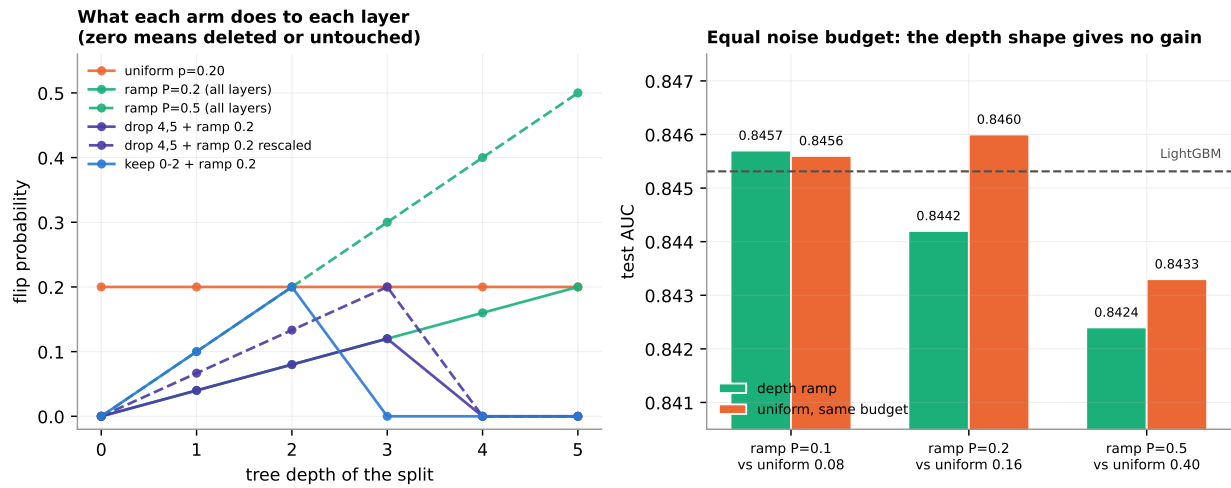


Figure 3: Left: the flip probability each arm applies at each depth; zero means the layer was deleted or deliberately untouched. Right: each depth ramp beside the uniform arm carrying the same average flip rate. At matched budget the ramp never wins.

7 Observations

1. **Test AUC is almost completely insensitive to everything tried.** Across all twenty-one configurations it spans 0.8424 to 0.8464, a range of 0.0040, against a baseline of 0.8457. For context, the standard error of an AUC estimated on 2400 rows is approximately 0.009, so no arm is distinguishable from the baseline in absolute terms. Thirteen of the eighteen configurations sit at or above the LightGBM ceiling.
2. **Overfitting, by contrast, is highly controllable.** The gap ranges from 0.0972 at the baseline to 0.0092 under the heaviest ramp, a reduction of **90.5%**. The relationship with the noise level is monotone: gaps of 0.0972, 0.0722, 0.0551, 0.0359 and 0.0130 for uniform flip rates of 0, 0.05, 0.10, 0.20 and 0.40.
3. **Deletion is the most efficient regulariser.** Keeping only depths 0–2 uses **700 of 5,401 bits**, a 7.7-fold reduction, and yields test AUC 0.8460 (baseline 0.8457) with the gap cut from 0.0972 to 0.0240, a **75.3% reduction**. It is also the fastest arm to fit, 3.3 s against 48.7 s for the baseline. No noise is required to achieve this.
4. **The depth shape of the noise provides no benefit at matched budget.** Comparing each ramp with the uniform arm of the same average rate: at $\bar{p} \approx 0.08$, 0.8457 versus 0.8456; at $\bar{p} \approx 0.16$, 0.8442 versus 0.8460; at $\bar{p} \approx 0.40$, 0.8424 versus 0.8433. The ramp is marginally behind in two of the three pairs and level in the third. The hypothesis that deeper splits deserve more perturbation is therefore *not* supported once the total amount of noise is controlled for.
5. **The combined design works, and works through deletion.** Dropping depths 4 and 5 and ramping the survivors gives test AUC 0.8459 with a gap of 0.0280, a 71.2% reduction, using 1480 bits. Decomposing it: deletion alone accounts for a gap of 0.0401 and adding the ramp brings a further 0.0121. Both components help, deletion more than the ramp.
6. **Five arms fall below the tree ceiling, and four of them contain a depth ramp.** They are ramp $P = 0.5$ (0.8424), uniform $p = 0.40$ (0.8433), keep 0–2 with a ramp (0.8437), ramp $P = 0.2$ (0.8442) and drop 4,5 with the rescaled ramp (0.8451). Grouping the configurations by whether they contain a ramp at all: the seven ramp arms average 0.8447 with four below the ceiling, while the eleven non-ramp arms average 0.8456 with one below. The two extreme-noise arms also have the two smallest gaps, which shows the gap can be pushed past the useful point: there the penalty is suppressing signal rather than memorisation.
7. **The raw features remain redundant.** Adding the ten original columns to the full encoding changes test AUC from 0.8457 to 0.8454 and the gap from 0.0972 to 0.0975. This is the third independent confirmation that the encoding subsumes the raw features.
8. **The measurements are stable.** Refitting three arms three times each with fresh noise gave test-AUC spreads of 0.0001, 0.0001 and 0.0002. The ordering between arms is therefore reproducible with respect to the noise draw, even though the absolute differences are small.

8 Key findings

1. **Overfitting in this model can be reduced by more than tenfold at no cost in test accuracy.** This confirms and quantifies the premise of this round. The gap is not a symptom that needs curing in order to improve generalisation; it can be removed almost entirely while test AUC does not move.
2. **The recommended configuration is to keep only the three shallowest tree layers.** Test AUC 0.8460 versus the LightGBM ceiling of 0.8453, overfitting gap 0.0240 instead of 0.0972, and 700 features instead of 5401. It requires no noise, no extra hyper-parameter and no stochastic training, and it fits about fifteen times faster. If a slightly tighter gap is wanted

at the same accuracy, dropping depths 4 and 5 and ramping the survivors gives 0.8459 with a gap of 0.0280.

3. **Depth-targeted perturbation is not justified by these results.** Once the total noise budget is matched, uniform flips do as well or slightly better, and the pattern holds when the configurations are simply grouped: the seven arms containing a ramp average 0.8447 in test AUC with four falling below the tree ceiling, against 0.8456 and one for the eleven arms without one. The intuition that deep splits are less reliable is correct as a description of the encoding (they are fitted on roughly ten times fewer rows), and *deleting* them is indeed harmless, but *graduating the noise by depth* adds nothing over flipping everything equally and is mildly worse here.
4. **Test performance on this dataset is saturated and set by the shallow tree structure.** Seven hundred bits from the top three layers extract the full predictive content; the remaining 4701 bits contribute nothing to generalisation and only provide capacity to memorise. Together with the earlier finding that a linear model matches a 2.44-million-parameter network, the picture is consistent: the quantity that determines accuracy here is the tree structure in the encoding, not the model on top of it and not the amount of regularisation applied to it.
5. **Practical consequence for the paper.** The method can be stated as a logistic regression on a shallow, out-of-bag-honest tree encoding. It is convex, has one hyper-parameter, trains in a few seconds, matches gradient boosting on this dataset, and overfits four times less than the full encoding. The next step required before any general claim is to repeat this on the other clean datasets in the suite, since every number in this report comes from a single dataset and a single split.

Reproduction.

Shared options for every arm:

```
run_feature_reg.py --task 361055 --C 0.01 --views tree --encoding oob --noise-copies 3
```

Per-arm flag added to the above:

```
--flip-uniform  --flip-ramp  --flip-ramp-rel  
--drop-layers  --keep-layers  --repeats
```

Driver notebook: `colab_feature_reg.ipynb`. All runs on CPU, random seed 0. Wall-clock time per fit ranged from 3.3s (700 bits, no noise) to 181.7s (5,401 bits, three noisy copies).